MEMORIA TÉCNICA: EVALUAT.IO

Autores: [Nombre del Alumno]

Fecha: Febrero 2026

Repositorio: `github.com/usuario/evaluatio`

Índice

1 Introducción y Contexto **3**
1.1 Antecedentes y Definición del Problema 3
1.2 Estado del Arte y Justificación 3
2 Análisis de Requisitos **4**
2.1 Requisitos Funcionales (RF) 4
2.2 Requisitos No Funcionales (RNF) 4
3 Planificación y Gestión del Tiempo **5**
3.1 Desglose de Tareas y Estimaciones (Backlog) 5
3.2 Cronograma de Trabajo (Gantt) 6
4 Análisis Funcional y Flujos **7**
4.1 Diagrama de Flujo (User Flow) 7
4.2 Interfaz de Usuario (UI) 7
5 Arquitectura y Modelo de Datos **8**
5.1 Diagrama de Clases 8
5.2 Diseño de la API (Endpoints Principales) 9
5.3 Stack Tecnológico y Justificación 9
6 Retos Técnicos y Soluciones **10**
6.1 El Reto: Asignación de Pares Aleatoria 10
6.2 El Reto: Rúbricas Dinámicas 10
7 Pruebas de Calidad (QA) **11**
8 Conclusiones y Futuros Pasos **12**

1 Introducción y Contexto

1.1 Antecedentes y Definición del Problema

En el sector educativo actual, y específicamente en formaciones intensivas tipo *bootcamp*, la evaluación continua es crítica. Sin embargo, la metodología de **evaluación cruzada** (*peer-review*), pese a su alto valor pedagógico, se enfrenta a graves problemas logísticos:

- **Fragmentación:** El proceso suele depender de herramientas desconectadas (Google Forms para recoger datos, Excel para calcular notas, Email para enviar feedback).
- **Falta de Privacidad:** El uso de hojas de cálculo compartidas expone a menudo las calificaciones o comentarios entre alumnos de forma accidental.
- **Carga Administrativa:** Se estima que los docentes invierten hasta un 30% de su tiempo en la gestión manual de estas notas en lugar de en la corrección cualitativa.

1.2 Estado del Arte y Justificación

Antes de abordar el desarrollo, se analizaron las soluciones existentes en el mercado:

- **LMS Tradicionales (Moodle/Canvas):** Ofrecen módulos de taller, pero su configuración es excesivamente compleja y rígida para ciclos de evaluación rápidos (sprints semanales).
- **Hojas de Cálculo:** Son flexibles pero propensas a errores humanos (fórmulas rotas) y carecen de control de acceso granular.
- **Formularios Web (Google Forms/Typeform):** Permiten recoger datos, pero no automatizan la asignación aleatoria de pares ni el cálculo ponderado de notas.

Nuestra Propuesta: Evaluat.io nace para cubrir este hueco. Es una solución vertical (SaaS) diseñada específicamente para la evaluación por rúbricas. A diferencia de un LMS generalista, Evaluat.io prioriza la agilidad: permite crear una sesión de evaluación, asignar pares aleatoriamente y visualizar resultados en minutos, garantizando la integridad de los datos y el anonimato necesario para una evaluación objetiva.

2 Análisis de Requisitos

Para garantizar la viabilidad del MVP en el plazo de dos semanas, se establecieron los siguientes requisitos formales.

2.1 Requisitos Funcionales (RF)

El sistema debe permitir las siguientes interacciones clave:

- **RF-01 Gestión de Usuarios (RBAC):** El sistema diferenciará estrictamente entre roles de 'Profesor' (creador) y 'Alumno' (evaluador).
- **RF-02 Rúbricas Dinámicas:** El profesor debe poder definir criterios de evaluación personalizados y sus pesos para cada tarea (ej: "Código Limpio: 40%", "Funcionalidad: 60%").
- **RF-03 Asignación Inteligente:** El sistema debe distribuir automáticamente las tareas entre los alumnos mediante un algoritmo que garantice que nadie se evalúa a sí mismo.
- **RF-04 Evaluación Ciega:** El alumno evaluador no podrá ver las notas otorgadas por otros compañeros para evitar sesgos (*anchoring bias*).
- **RF-05 Feedback Consolidado:** El alumno evaluado tendrá acceso a un dashboard donde verá su nota final calculada automáticamente basada en la ponderación definida.

2.2 Requisitos No Funcionales (RNF)

Restricciones técnicas y de calidad que rigen el desarrollo:

- **RNF-01 Integridad de Datos:** Una vez enviada una evaluación, esta queda bloqueada y no puede ser modificada por el alumno para evitar fraudes.
- **RNF-02 Seguridad:** Todas las rutas de escritura deben estar protegidas mediante validación de token (JWT). Un alumno no puede forzar una petición para evaluar una tarea que no tiene asignada.
- **RNF-03 Usabilidad (Mobile First):** La interfaz de evaluación debe ser totalmente responsiva, permitiendo a los alumnos corregir tareas desde dispositivos móviles o tablets.
- **RNF-04 Escalabilidad de Base de Datos:** El modelo relacional debe soportar la integridad referencial (Foreign Keys) para evitar registros huérfanos si se borra un usuario o tarea.

3 Planificación y Gestión del Tiempo

Para garantizar la entrega del MVP en el plazo de 10 días laborables, se realizó un desglose de tareas y una estimación de esfuerzo en horas efectivas.

3.1 Desglose de Tareas y Estimaciones (Backlog)

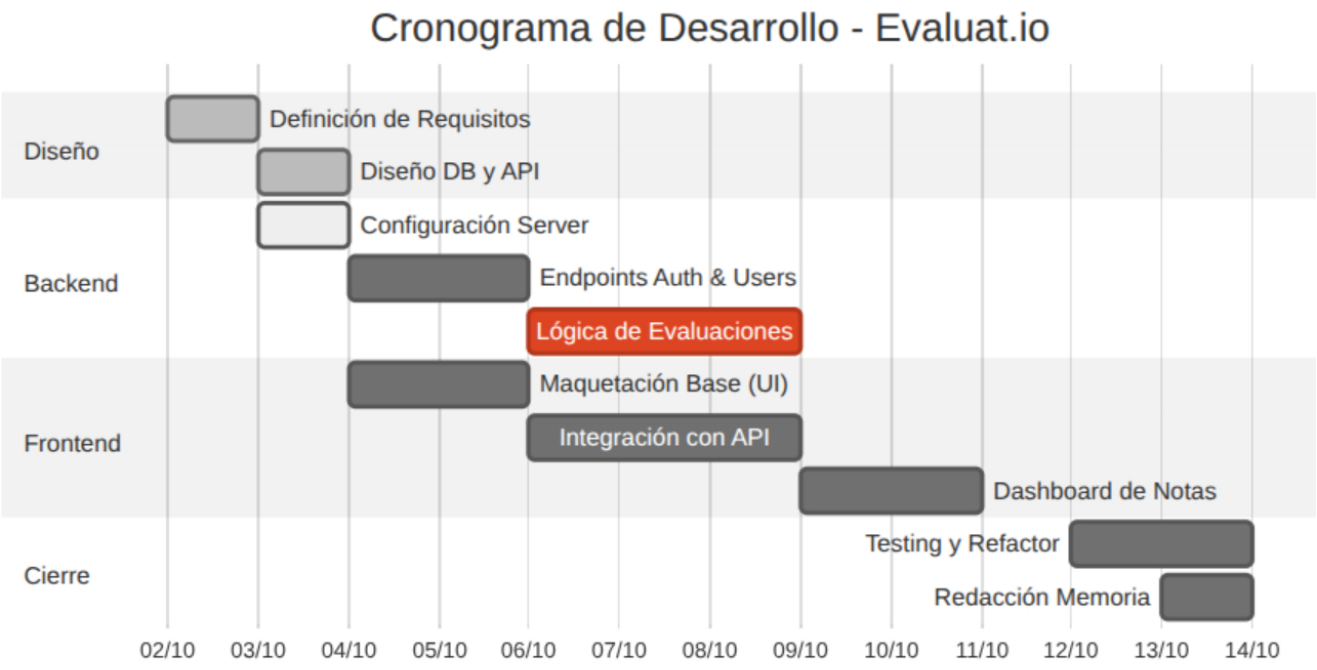
Se ha estimado una capacidad de trabajo de **6 horas efectivas de código al día** (dejando 2h para reuniones, documentación y bloqueos imprevistos).

Fase	Tarea (Ticket)	Descripción Técnica	Estimación	Dependencias
Diseño	**Definición y DB**	Diagramas E-R, User Flow y contrato de API (JSON).	**8h** (1.5 días)	-
Back	**Setup Inicial**	Configuración Express, Cors, Dotenv y conexión a Postgres.	**4h**	Diseño DB
Back	**Auth System**	Endpoints `/register`, `/login`, Middleware JWT y Hashing.	**10h** (1.5 días)	Setup
Back	**Core Logic**	CRUD de Tareas y Algoritmo `Fisher-Yates` para asignación de pares.	**16h** (2.5 días)	Auth
Front	**UI Base**	Configuración React, Router y Componentes base (Navbar, Layout).	**8h** (1.5 días)	Diseño
Front	**Integración**	Consumo de API con `fetch/axios`, gestión de estados y formularios.	**14h** (2.5 días)	Endpoints Back
Front	**Dashboard**	Visualización de notas y cálculo de medias en cliente.	**10h** (1.5 días)	Lógica Back
Cierre	**QA & Fixes**	Testing manual de flujos críticos (Happy Path) y corrección de bugs.	**8h** (1.5 días)	Integración
Docs	**Memoria**	Redacción de documentación técnica y limpieza de repo.	**5h** (1 día)	-

> **Nota sobre la estimación:** Se ha aplicado un "Factor de Seguridad" del 20% sobre las horas estimadas para cubrir imprevistos (bugs complejos o deuda técnica).

3.2 Cronograma de Trabajo (Gantt)

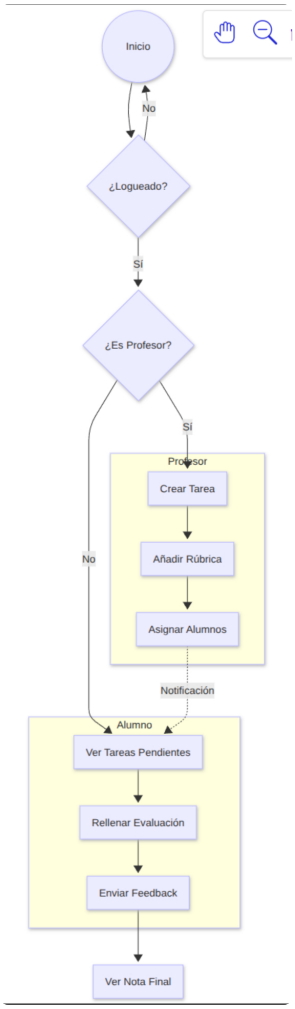
Basado en las estimaciones anteriores y las dependencias entre Backend y Frontend, se trazó la siguiente hoja de ruta visual para coordinar el desarrollo paralelo.



4 Análisis Funcional y Flujos

El sistema se basa en dos actores principales con privilegios diferenciados. A continuación se detalla el flujo principal de uso de la aplicación: desde que un profesor crea una tarea hasta que el alumno recibe su nota.

4.1 Diagrama de Flujo (User Flow)



4.2 Interfaz de Usuario (UI)

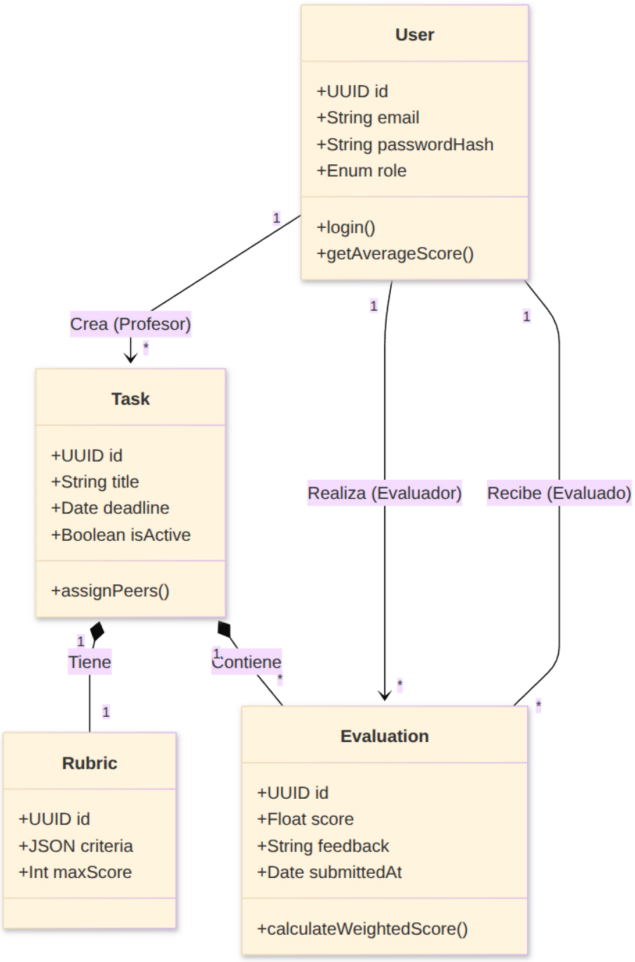
Se incluyen capturas del producto final para validar el cumplimiento de los requisitos de diseño.

- Vista de Evaluación (Mobile): Muestra cómo el alumno puede puntuar la rúbrica fácilmente desde el móvil.
- Dashboard del Profesor (Desktop): Vista tabular para gestionar las entregas.(Aquí el alumno insertaría 2 capturas de pantalla)

5 Arquitectura y Modelo de Datos

5.1 Diagrama de Clases

Para representar la estructura del código y las entidades del dominio, se ha diseñado el siguiente modelo. Destaca la relación entre `Task`, `Rubric` y `Evaluation`, que permite que una misma tarea sea evaluada múltiples veces por distintos usuarios.



5.2 Diseño de la API (Endpoints Principales)

Se ha diseñado una API RESTful siguiendo las convenciones de nomenclatura de recursos.

Método	Endpoint	Descripción	Protegido (JWT)
`POST`	`/api/auth/login`	Autenticación y retorno de Token.	No
`GET`	`/api/tasks`	Lista las tareas asignadas al usuario.	Sí
`POST`	`/api/tasks`	Crea una nueva tarea (Solo Profesores).	Sí
`POST`	`/api/evaluations`	Envía una evaluación completa.	Sí

5.3 Stack Tecnológico y Justificación

- **Backend:** Node.js + Express. Se eligió por su arquitectura no bloqueante, ideal para manejar múltiples peticiones de evaluación simultáneas.
- **Base de Datos:** PostgreSQL. Necesario para mantener la integridad referencial entre Usuarios y Evaluaciones (evitar que una evaluación exista sin un usuario asociado).
- **Frontend:** React + Context API. Para gestionar el estado global de la sesión del usuario y las rúbricas dinámicas.

6 Retos Técnicos y Soluciones

6.1 El Reto: Asignación de Pares Aleatoria

Situación: Uno de los requisitos era que el profesor pudiera pulsar un botón y que los alumnos se asignaran aleatoriamente para evaluarse entre sí, evitando que un alumno se evalúe a sí mismo.

Investigación: Inicialmente se intentó hacer en el Frontend, pero resultaba inseguro y lento con muchos alumnos.

Solución Implementada: Se creó un algoritmo en el Backend (Fisher-Yates Shuffle) que mezcla el array de alumnos y crea los pares en base de datos en una sola transacción.

```
// Ejemplo simplificado del algoritmo usado
function assignPeers(students) {
    let shuffled = [...students].sort(() => 0.5 - Math.random());
    return shuffled.map((student, index) => {
        let evaluator = student;
        let target = shuffled[(index + 1) % shuffled.length]; // Round
Robin
        return { evaluator, target };
    });
}
```

6.2 El Reto: Rúbricas Dinámicas

Situación: Cada tarea puede tener una rúbrica con diferente número de criterios y pesos.

Solución: Se utilizó un campo `JSONB` en PostgreSQL para almacenar la estructura de la rúbrica, permitiendo flexibilidad total sin necesidad de alterar el esquema de la base de datos para cada nueva tarea.

7 Pruebas de Calidad (QA)

Antes del cierre del proyecto, se realizó una batería de pruebas manuales para validar los flujos críticos (Happy Path) y la gestión de errores (Edge Cases).

Caso de Prueba	Resultado Esperado	Estado
Login Incorrecto	El sistema debe mostrar "Credenciales Inválidas" sin revelar si el email existe.	✅ Pasado
Auto-Evaluación	Un alumno intenta evaluar su propia tarea modificando la URL/ID. La API devuelve 403 Forbidden.	✅ Pasado
Cálculo de Nota	Verificar que (Profesor * 0.6) + (Peers * 0.4) da el resultado exacto.	✅ Pasado

8 Conclusiones y Futuros Pasos

El desarrollo de **Evaluat.io** ha permitido consolidar conocimientos sobre arquitecturas MVC y gestión de bases de datos relacionales. El sistema es funcional y cumple con los requisitos de evaluación académica.

Mejoras para la v2.0:

1. Implementar WebSockets para ver en tiempo real quién ha terminado de evaluar.
2. Añadir exportación de notas a CSV/Excel para los profesores.
3. Mejorar la accesibilidad (a11y) de los formularios de evaluación.